
INTRODUCTION TO JAVASCRIPT, VARIABLES AND DATA TYPES

What is JavaScript?

- JavaScript (JS) is a high-level, interpreted programming language used to make web pages interactive.
- It is one of the core technologies of the web, alongside HTML (structure) and CSS (style).
- **Features:**
 - **Lightweight** → Efficient and fast for web use.
 - **Dynamically typed** → No need for type declarations, flexible but error-prone.
 - **Multi-paradigm** → Supports procedural, functional, and OOP programming styles.
 - **Runs everywhere** → Browser (client-side) + Node.js (server-side).

1. Lightweight

- JavaScript is designed to be **minimal and efficient**, not heavy like Java or C++.
- It has a **small set of core features**, and everything else can be added using libraries or frameworks.
- Because it's lightweight, it loads quickly in browsers, making web pages interactive without consuming too much memory or CPU.
- **Example:**

A small snippet of JavaScript can add interactivity to a webpage:

```
<button onclick="alert('Hello!')">Click Me</button>
```

2. Dynamically Typed

- In JavaScript, **you don't need to declare the type** of a variable.
- The type is decided at runtime (while the program is running).

- This makes coding faster but can also lead to unexpected errors.
- **Example:**

```
let x = 10;    // x is a Number
```

```
x = "Hello";  // now x is a String
```

In Java or C++, this would cause an error because variables have fixed types, but in JavaScript, it's totally allowed.

3. Multi-Paradigm

JavaScript supports multiple programming styles, which makes it flexible:

- **Procedural (step-by-step instructions):**

Ex:

```
function greet() {  
    console.log("Hello, World!");  
}  
  
greet();
```

- **Functional** (functions as first-class citizens, higher-order functions):

Ex: const numbers = [1, 2, 3, 4];

```
const doubled = numbers.map(n => n * 2); // functional style
```

```
console.log(doubled); // [2, 4, 6, 8]
```

- **Object-Oriented** (using classes and objects):

Ex:

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
  sayHi() {  
    console.log(`Hi, I'm ${this.name}`);  
  }  
}  
  
const p1 = new Person("Alice");  
  
p1.sayHi(); // Hi, I'm Alice
```

This flexibility means developers can **choose the style** that best fits their project.

4. Runs on Both Client-Side and Server-Side

- **Client-side (Browser):** JavaScript makes websites interactive. For example, validating a form before sending it to the server.
- **Server-side (Node.js):** With the advent of Node.js, JavaScript can also run on the backend to handle databases, APIs, authentication, etc.

Client-side Example (browser):

```
document.getElementById("btn").addEventListener("click", () => {  
  alert("Button clicked!");  
});
```

Server-side Example (Node.js):

// Run with Node.js

```
const http = require('http');  
  
http.createServer((req, res) => { res.write("Hello from server!");  
res.end(); }).listen(3000);
```

Here, the same language (JavaScript) can be used for **both frontend and backend**, reducing the need to learn two different languages.

Role of JS in Web Development

- **HTML** defines structure → headings, paragraphs, buttons.
- **CSS** defines presentation → colors, fonts, layouts.
- **JavaScript** adds behavior:
 - Responds to user actions (clicks, inputs, hovers).
 - Validates form inputs.
 - Fetches data without reloading the page (AJAX, Fetch API).
 - Creates animations, sliders, and interactive UI components.

How JS Runs in the Browser

- **JS Engine:** A program inside browsers (e.g., V8 in Chrome, SpiderMonkey in Firefox) that executes JS code.
- **Execution Context:**
 - Global Execution Context (default environment for running code).
 - Function Execution Context (created when a function runs).
 - **Call Stack:** Stores function calls in order. JS is **single-threaded** but uses **event loop** for async operations.

Adding JS to HTML

There are three main ways:

1. Internal Script

Ex:

```
<script>  
  
  console.log("Hello from Internal JS");  
  
</script>
```

2. External Script (Recommended)

Keeps code separate and reusable.

Ex: `<script src="app.js"></script>`

3. Inline Script (Not recommended for large projects)

```
<button onclick="alert('Button clicked!')">Click Me</button>
```

Console & Debugging Basics

- Use **console.log()** to print messages.
- Other console methods:
 - **console.error("Error message")**
 - **console.warn("Warning message")**
 - **console.table([1, 2, 3])**

Debugging tools:

- Browser **DevTools** (Inspect → Console tab).
- **Breakpoints** to pause and inspect code execution.

Variables in JavaScript

What is a Variable?

A variable is a **container for storing data**.

Syntax:

```
let name = "John";
```

Declaring Variables

1. **var** – Old way, function-scoped, hoisted.

Ex: var x = 10;

2. **let** – Modern, block-scoped, allows reassignment.

Ex: let y = 20;

3. **const** – Block-scoped, **cannot be reassigned**.

Ex: const PI = 3.14;

Scope: Region where a variable is accessible.

- **Global Scope:** Accessible anywhere.
- **Block Scope:** Exists inside `{ }` when declared with **let** or **const**.

Global vs Block Scope

Ex1: `var globalVar = "I am global";`
`{`
 `let blockVar = "I am block scoped";`
 `console.log(blockVar); // works`
`}`
`console.log(globalVar); // works`
`// console.log(blockVar); // Error`

Ex2:

// Using var

```
if (true) {  
    var x = 10; // function-scoped or global-scoped  
}  
  
console.log(x); //  works → 10
```

// Using let

```
if (true) {  
    let y = 20; // block-scoped  
}  
  
console.log(y); //  ReferenceError: y is not defined
```

Hoisting:

Hoisting means **JavaScript moves declarations to the top of their scope (before code execution)**.

- **var** is hoisted with **undefined** value.
- **let** and **const** are hoisted but kept in **Temporal Dead Zone (TDZ)** until initialized.

Ex:

Example with **var**:

```
console.log(a); // ✓ undefined (hoisted)
```

```
var a = 10;
```

```
console.log(a); // 10
```

Here, JavaScript internally treats it like:

```
var a;      // declaration hoisted to top
```

```
console.log(a); // undefined
```

```
a = 10;     // initialization happens later
```

```
console.log(a); // 10
```

Example with **let / const**:

```
console.log(b); // ✗ ReferenceError
```

```
let b = 20;
```

With **let** and **const**, the declaration is also hoisted but kept in a **Temporal Dead Zone (TDZ)** until the line where it's declared. That's why accessing it before declaration throws an error.

Best Practices for Variable Declaration

- Use **const** by default.
- Use **let** only when you plan to reassign.
- Avoid **var** in modern code.
Use meaningful names (**userName**, not **x**).

Data Types in JavaScript

Primitive Data Types

1. String:

Ex: let name = "Alice";

2. Number:

Ex:

let age = 25;

let price = 99.99;

3. Boolean:

Ex: let isLoggedIn = true; //true or false

4. Undefined:

Ex: let x;

console.log(x); // undefined

5. Null:

Ex: `let empty = null; // used for objects`

6. BigInt (for very large numbers):

Ex:

`let big = 123456789012345678901234567890n;`

Non-Primitive Data Types

1. Objects (key-value pairs)

`let person = { name: "Bob", age: 30 };`

2. Arrays (ordered lists)

`let fruits = ["apple", "banana", "cherry"];`

3. Functions (first-class citizens)

`function greet() { console.log("Hello!"); }`

typeof Operator

Checks the type of a variable:

Ex:

```
console.log(typeof "hello"); // string
console.log(typeof 42);      // number
console.log(typeof true);    // boolean
console.log(typeof undefined); // undefined
console.log(typeof null);    // object (quirk in JS)
console.log(typeof {});      // object
console.log(typeof []);      // object (arrays are special objects)
console.log(typeof function(){}); // function
```